

WASAC/REG Utility Billing System

Complete API Documentation

Version: 1.0.0

Base URL: `http://localhost:8080`

Date: June 2026

Stack: Java 21, Spring Boot 3.x, PostgreSQL, JWT, Resend Email

Table of Contents

1. [Getting Started](#)
2. [Authentication](#)
3. [System Roles & Permissions](#)
4. [Customer Management](#)
5. [Meter Management](#)
6. [Meter Reading Capture](#)
7. [Tariff & Configuration](#)
8. [Billing Engine](#)
9. [Payment Processing](#)
10. [Notification Logs](#)
11. [Complete Workflow Example](#)
12. [Error Reference](#)
13. [Database Schema](#)
14. [Testing with Swagger UI](#)
15. [Environment Variables](#)

1. Getting Started

1.1 Prerequisites

Before using the API, ensure you have:

- PostgreSQL running on `localhost:5432`
- Database `utility_billing` created
- Java 21 installed
- Maven installed
- Environment variables configured (or use defaults)

1.2 Environment Variables

Variable	Default	Description
<code>DB_USERNAME</code>	<code>postgres</code>	PostgreSQL username
<code>DB_PASSWORD</code>	<code>root</code>	PostgreSQL password
<code>JWT_SECRET</code>	<code>42371bdd...</code>	JWT signing key (256-bit minimum)
<code>RESEND_API_KEY</code>	<code>re_QeFD8Cq2...</code>	Resend email API key

1.3 Running the Application

```
mvn spring-boot:run
```

The application starts on **port 8080**.

1.4 Available URLs

Resource	URL
Swagger UI	<code>http://localhost:8080/swagger-ui.html</code>
OpenAPI JSON	<code>http://localhost:8080/v3/api-docs</code>

Resource	URL
API Base	<code>http://localhost:8080/api/</code>

2. Authentication

All endpoints except `/api/auth/**` require a valid **JWT Bearer token** in the `Authorization` header.

```
Authorization: Bearer <your-jwt-token>
```

2.1 POST /api/auth/register

Description: Register a new customer account. Default role is `ROLE_CUSTOMER`. Status is `ACTIVE`.

Access: Public (no authentication required)

Request Headers

```
Content-Type: application/json
```

Request Body

```
{
  "fullNames": "Jean Pierre Uwimana",
  "email": "jean.pierre@example.com",
  "phoneNumber": "+250788123456",
  "password": "SecurePass123"
}
```

Validation Rules

Field	Rules
<code>fullNames</code>	Required, 2-255 characters
<code>email</code>	Required, valid email format, max 255 chars
<code>phoneNumber</code>	Optional, 8-15 digits, may start with <code>+</code>

Field	Rules
password	Required, minimum 6 characters

Success Response (200 OK)

```
{
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "type": "Bearer",
  "email": "jean.pierre@example.com",
  "roles": ["ROLE_CUSTOMER"]
}
```

Error Response (400 Bad Request) - Duplicate Email

```
{
  "timestamp": "2026-06-05T10:30:00",
  "status": 400,
  "message": "Email already in use",
  "path": "/api/auth/register"
}
```

Error Response (400 Bad Request) - Validation Fail

```
{
  "timestamp": "2026-06-05T10:30:00",
  "status": 400,
  "message": "Validation failed: {fullNames=Full names must be between 2 and 255 characters}",
  "path": "/api/auth/register"
}
```

2.2 POST /api/auth/login

Description: Authenticate and receive a JWT token valid for 24 hours.

Access: Public

Request Body

```
{
  "email": "admin@wasac.rw",
  "password": "Admin123"
}
```

Success Response (200 OK)

```
{
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "type": "Bearer",
  "email": "admin@wasac.rw",
  "roles": ["ROLE_ADMIN", "ROLE_OPERATOR", "ROLE_FINANCE"]
}
```

Error Response (401 Unauthorized)

```
{
  "timestamp": "2026-06-05T10:32:00",
  "status": 401,
  "message": "Invalid email or password",
  "path": "/api/auth/login"
}
```

cURL Example

```
curl -X POST http://localhost:8080/api/auth/login \
-H "Content-Type: application/json" \
-d '{
  "email": "admin@wasac.rw",
  "password": "Admin123"
}'
```

3. System Roles & Permissions

Role	Permissions
ROLE_ADMIN	Configure tariffs, approve bills, manage users, manage customers, manage meters, view all data
ROLE_OPERATOR	Capture meter readings, generate bills

Role	Permissions
ROLE_FINANCE	Approve bills, record payments, view all bills and payments
ROLE_CUSTOMER	View own bills and payment history only

Default Admin Account (Seeded on startup)

Email: admin@wasac.rw
 Password: Admin123
 Roles: ADMIN, OPERATOR, FINANCE

4. Customer Management

Base URL: /api/customers

Required Role: ROLE_ADMIN

4.1 POST /api/customers

Description: Create a new utility customer. National ID must be unique.

Request Headers

Content-Type: application/json
 Authorization: Bearer <token>

Request Body

```
{
  "fullNames": "Marie Claire Mukamana",
  "nationalId": "119908001234567",
  "email": "marie.mukamana@example.com",
  "phoneNumber": "+250788999888",
  "address": "KG 123 St, Kigali, Rwanda",
  "status": "ACTIVE"
}
```

Validation Rules

Field	Rules
fullNames	Required, 2-255 characters
nationalId	Required, alphanumeric, 5-20 characters, unique
email	Required, valid email format
phoneNumber	Optional, 8-15 digits, may start with +
address	Optional, max 500 characters
status	Optional, must be ACTIVE or INACTIVE

Success Response (200 OK)

```
{
  "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "fullNames": "Marie Claire Mukamana",
  "nationalId": "119908001234567",
  "email": "marie.mukamana@example.com",
  "phoneNumber": "+250788999888",
  "address": "KG 123 St, Kigali, Rwanda",
  "status": "ACTIVE",
  "createdAt": "2026-06-05T10:00:00",
  "updatedAt": "2026-06-05T10:00:00"
}
```

Error Response (400) - Duplicate National ID

```
{
  "timestamp": "2026-06-05T10:00:00",
  "status": 400,
  "message": "Customer with national ID already exists: 119908001234567",
  "path": "/api/customers"
}
```

cURL Example

```
curl -X POST http://localhost:8080/api/customers \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <token>" \
-d '{
  "fullNames": "Marie Claire Mukamana",
  "nationalId": "119908001234567",
  "email": "marie.mukamana@example.com",
  "phoneNumber": "+250788999888",
  "address": "KG 123 St, Kigali, Rwanda"
}'
```

4.2 GET /api/customers

Description: List all customers with pagination.

Query Parameters

Parameter	Example	Description
page	0	Page number (0-indexed)
size	10	Items per page
sort	fullNames,asc	Sort field and direction

Success Response (200 OK)

```
{
  "content": [
    {
      "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
      "fullNames": "Marie Claire Mukamana",
      "nationalId": "119908001234567",
      "email": "marie.mukamana@example.com",
      "phoneNumber": "+250788999888",
      "address": "KG 123 St, Kigali, Rwanda",
      "status": "ACTIVE",
      "createdAt": "2026-06-05T10:00:00",
      "updatedAt": "2026-06-05T10:00:00"
    }
  ],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 10
  },
  "totalElements": 1,
  "totalPages": 1
}
```

cURL Example

```
curl "http://localhost:8080/api/customers?page=0&size=10&sort=fullNames,asc" \
-H "Authorization: Bearer <token>"
```

4.3 GET /api/customers/{id}

Description: Get a single customer by UUID.

Path Parameters

Parameter	Description
id	Customer UUID (e.g., <code>a1b2c3d4-e5f6-7890-abcd-ef1234567890</code>)

Success Response (200 OK)

Same structure as the customer object in POST response.

Error Response (404) - Not Found

```
{
  "timestamp": "2026-06-05T10:00:00",
  "status": 404,
  "message": "Customer not found with id : 'a1b2c3d4-e5f6-7890-abcd-ef1234567890'",
  "path": "/api/customers/a1b2c3d4-e5f6-7890-abcd-ef1234567890"
}
```

cURL Example

```
curl http://localhost:8080/api/customers/a1b2c3d4-e5f6-7890-abcd-ef1234567890 \
-H "Authorization: Bearer <token>"
```

4.4 PUT /api/customers/{id}

Description: Update customer details. National ID uniqueness is enforced.

Request Body

```
{
  "fullNames": "Marie Claire Mukamana Updated",
  "nationalId": "119908001234567",
  "email": "marie.updated@example.com",
  "phoneNumber": "+250788999888",
  "address": "KN 456 Ave, Kigali, Rwanda",
  "status": "ACTIVE"
}
```

Success Response (200 OK)

Returns updated customer object.

4.5 DELETE /api/customers/{id}

Description: Soft-delete a customer by setting status to **INACTIVE**. Inactive customers cannot receive new bills.

Success Response (204 No Content)

No response body.

cURL Example

```
curl -X DELETE http://localhost:8080/api/customers/a1b2c3d4-e5f6-7890-abcd-ef1234567890 \
-H "Authorization: Bearer <token>"
```

5. Meter Management

Base URL: `/api/meters`

Required Role: `ROLE_ADMIN`

5.1 POST /api/meters

Description: Register a new utility meter. `meterNumber` must be globally unique. One customer can have multiple meters.

Request Body

```
{
  "meterNumber": "WTR-2026-001",
  "meterType": "WATER",
  "installationDate": "2026-01-15",
  "status": "ACTIVE",
  "customerId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
}
```

Validation Rules

Field	Rules
<code>meterNumber</code>	Required, 1-100 characters, globally unique
<code>meterType</code>	Required, must be <code>WATER</code> or <code>ELECTRICITY</code>
<code>installationDate</code>	Required
<code>status</code>	Optional, must be <code>ACTIVE</code> or <code>INACTIVE</code>

Field	Rules
customerId	Required, valid customer UUID

Success Response (200 OK)

```
{
  "id": "b2c3d4e5-f6a7-8901-bcde-f23456789012",
  "meterNumber": "WTR-2026-001",
  "meterType": "WATER",
  "installationDate": "2026-01-15",
  "status": "ACTIVE",
  "customerId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "customerName": "Marie Claire Mukamana",
  "createdAt": "2026-06-05T10:05:00",
  "updatedAt": "2026-06-05T10:05:00"
}
```

Error Response (400) - Duplicate Meter Number

```
{
  "timestamp": "2026-06-05T10:05:00",
  "status": 400,
  "message": "Meter number already exists: WTR-2026-001",
  "path": "/api/meters"
}
```

cURL Example

```
curl -X POST http://localhost:8080/api/meters \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <token>" \
-d '{
  "meterNumber": "WTR-2026-001",
  "meterType": "WATER",
  "installationDate": "2026-01-15",
  "customerId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
}'
```

5.2 GET /api/meters

Description: List all meters with pagination.

Query Parameters

Same pagination pattern as customers: `page` , `size` , `sort` .

Success Response (200 OK)

Returns a paginated list of meter objects.

5.3 GET /api/meters/{id}

Description: Get a single meter by UUID.

5.4 PUT /api/meters/{id}

Description: Update meter details. Same validation rules as creation.

5.5 DELETE /api/meters/{id}

Description: Deactivate a meter by setting status to `INACTIVE` . Inactive meters cannot receive new readings.

6. Meter Reading Capture

Base URL: `/api/meter-readings`

Required Role: `ROLE_OPERATOR`

6.1 POST /api/meter-readings

Description: Capture a monthly meter reading. The system validates:

- Meter is **ACTIVE**
- Only **one reading per meter per month+year**

- `currentReading` must be **strictly greater** than `previousReading`
- `consumption` is auto-computed (`current` - `previous`)

Request Body

```
{
  "meterId": "b2c3d4e5-f6a7-8901-bcde-f23456789012",
  "previousReading": 1250.50,
  "currentReading": 1380.75,
  "readingDate": "2026-05-31",
  "readingMonth": 5,
  "readingYear": 2026
}
```

Validation Rules

Field	Rules
<code>meterId</code>	Required, valid meter UUID
<code>previousReading</code>	Required, zero or positive
<code>currentReading</code>	Required, zero or positive, must be > <code>previousReading</code>
<code>readingDate</code>	Required
<code>readingMonth</code>	Required, 1-12
<code>readingYear</code>	Required, 2000-2100

Success Response (200 OK)

```
{
  "id": "c3d4e5f6-a7b8-9012-cdef-345678901234",
  "meterId": "b2c3d4e5-f6a7-8901-bcde-f23456789012",
  "meterNumber": "WTR-2026-001",
  "previousReading": 1250.50,
  "currentReading": 1380.75,
  "consumption": 130.25,
  "readingDate": "2026-05-31",
  "readingMonth": 5,
  "readingYear": 2026,
  "createdAt": "2026-06-05T10:10:00",
  "updatedAt": "2026-06-05T10:10:00"
}
```

Error Response (400) - Inactive Meter

```
{
  "timestamp": "2026-06-05T10:10:00",
  "status": 400,
  "message": "Meter is not active. Readings can only be captured for active meters.",
  "path": "/api/meter-readings"
}
```

Error Response (400) - Duplicate Month/Year

```
{
  "timestamp": "2026-06-05T10:10:00",
  "status": 400,
  "message": "A reading for this meter already exists for the specified month and year.",
  "path": "/api/meter-readings"
}
```

Error Response (400) - Non-increasing Reading

```
{
  "timestamp": "2026-06-05T10:10:00",
  "status": 400,
  "message": "Current reading must be strictly greater than previous reading.",
  "path": "/api/meter-readings"
}
```

cURL Example

```
curl -X POST http://localhost:8080/api/meter-readings \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <operator-token>" \
-d '{
  "meterId": "b2c3d4e5-f6a7-8901-bcde-f23456789012",
  "previousReading": 1250.50,
  "currentReading": 1380.75,
  "readingDate": "2026-05-31",
  "readingMonth": 5,
  "readingYear": 2026
}'
```

6.2 GET /api/meter-readings

Description: List all meter readings with pagination.

6.3 GET /api/meter-readings/{id}

Description: Get a single meter reading by UUID.

7. Tariff & Configuration

Base URL: /api/tariffs , /api/service-charges , /api/tax-configs , /api/penalty-configs

Required Role: ROLE_ADMIN

7.1 POST /api/tariffs

Description: Create a new tariff. Versions auto-increment per meter type. New tariffs start as **INACTIVE** — activation is a separate step.

Tariff Types

- **FLAT:** Single price per unit (e.g., 300 FRW per m³)
- **TIER:** Progressive pricing bands (e.g., 0-50 units @ 200 FRW, 51-100 @ 250 FRW, etc.)

Request Body (FLAT Tariff)

```
{
  "meterType": "WATER",
  "tariffType": "FLAT",
  "unitPrice": 300.00,
  "effectiveDate": "2026-01-01",
  "description": "Standard water flat rate 2026"
}
```

Request Body (TIER Tariff - no unitPrice needed)

```
{
  "meterType": "WATER",
  "tariffType": "TIER",
  "effectiveDate": "2026-01-01",
  "description": "Progressive water tariff 2026"
}
```

Validation Rules

Field	Rules
meterType	Required, WATER or ELECTRICITY
tariffType	Required, FLAT or TIER
unitPrice	Required for FLAT, zero or positive
effectiveDate	Required
description	Required

Success Response (200 OK)

```
{
  "id": "d4e5f6a7-b8c9-0123-defa-456789012345",
  "meterType": "WATER",
  "tariffType": "FLAT",
  "unitPrice": 300.00,
  "effectiveDate": "2026-01-01",
  "version": 1,
  "status": "INACTIVE",
  "description": "Standard water flat rate 2026",
  "tiers": [],
  "createdAt": "2026-06-05T10:15:00",
  "updatedAt": "2026-06-05T10:15:00"
}
```

7.2 PUT /api/tariffs/{id}/activate

Description: Activate a tariff. This automatically **deactivates** the previous active tariff for the same meter type. Only one active tariff per meter type is allowed.

Path Parameters

Parameter	Description
<code>id</code>	Tariff UUID to activate

Success Response (200 OK)

Returns the activated tariff with `status: "ACTIVE"`.

cURL Example

```
curl -X PUT http://localhost:8080/api/tariffs/d4e5f6a7-b8c9-0123-defa-456789012345/activate \
-H "Authorization: Bearer <token>"
```

7.3 POST /api/tariffs/{id}/tiers

Description: Add a pricing tier to a TIER-type tariff. Cannot add tiers to FLAT tariffs.

Path Parameters

Parameter	Description
<code>id</code>	Parent tariff UUID

Request Body

```
{
  "tariffId": "d4e5f6a7-b8c9-0123-defa-456789012345",
  "minUnits": 0,
  "maxUnits": 50,
  "pricePerUnit": 200.00
}
```

Validation Rules

Field	Rules
<code>tariffId</code>	Required, valid tariff UUID
<code>minUnits</code>	Required, zero or positive

Field	Rules
maxUnits	Optional, zero or positive (null = unlimited)
pricePerUnit	Required, zero or positive

Success Response (200 OK)

Returns the updated tariff with the new tier included.

Error Response (400) - Not a TIER tariff

```
{
  "timestamp": "2026-06-05T10:15:00",
  "status": 400,
  "message": "Tiers can only be added to TIER type tariffs.",
  "path": "/api/tariffs/d4e5f6a7-b8c9-0123-defa-456789012345/tiers"
}
```

cURL Example

```
curl -X POST http://localhost:8080/api/tariffs/d4e5f6a7-b8c9-0123-defa-456789012345/tiers \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <token>" \
-d '{
  "tariffId": "d4e5f6a7-b8c9-0123-defa-456789012345",
  "minUnits": 0,
  "maxUnits": 50,
  "pricePerUnit": 200.00
}'
```

7.4 POST /api/service-charges

Description: Create a fixed service charge applied to bills of a specific meter type.

Request Body

```
{
  "name": "Monthly Maintenance Fee",
  "amount": 500.00,
  "meterType": "WATER",
  "effectiveDate": "2026-01-01"
}
```

Success Response (200 OK)

```
{
  "id": "e5f6a7b8-c9d0-1234-efab-567890123456",
  "name": "Monthly Maintenance Fee",
  "amount": 500.00,
  "meterType": "WATER",
  "effectiveDate": "2026-01-01",
  "status": "ACTIVE",
  "createdAt": "2026-06-05T10:20:00",
  "updatedAt": "2026-06-05T10:20:00"
}
```

7.5 POST /api/tax-configs

Description: Create a tax configuration (e.g., VAT). Rate is a decimal between 0 and 1.

Request Body

```
{
  "taxName": "VAT",
  "rate": 0.18,
  "effectiveDate": "2026-01-01"
}
```

Validation Rules

Field	Rules
taxName	Required
rate	Required, 0.0 to 1.0 (e.g., 0.18 = 18%)
effectiveDate	Required

Success Response (200 OK)

```
{
  "id": "f6a7b8c9-d0e1-2345-fabc-678901234567",
  "taxName": "VAT",
  "rate": 0.18,
  "effectiveDate": "2026-01-01",
  "status": "ACTIVE",
  "createdAt": "2026-06-05T10:25:00",
  "updatedAt": "2026-06-05T10:25:00"
}
```

7.6 POST /api/penalty-configs

Description: Create a late payment penalty rule.

Penalty Types

- **FIXED:** Flat amount (e.g., 1000 FRW)
- **PERCENTAGE:** Percentage of outstanding balance (e.g., 5%)

Request Body

```
{
  "penaltyType": "FIXED",
  "value": 1000.00,
  "gracePeriodDays": 15,
  "effectiveDate": "2026-01-01"
}
```

Validation Rules

Field	Rules
penaltyType	Required, FIXED or PERCENTAGE
value	Required, zero or positive
gracePeriodDays	Required, zero or positive
effectiveDate	Required

Success Response (200 OK)

```
{
  "id": "a7b8c9d0-e1f2-3456-abcd-789012345678",
  "penaltyType": "FIXED",
  "value": 1000.00,
  "gracePeriodDays": 15,
  "effectiveDate": "2026-01-01",
  "status": "ACTIVE",
  "createdAt": "2026-06-05T10:30:00",
  "updatedAt": "2026-06-05T10:30:00"
}
```

8. Billing Engine

Base URL: /api/bills
Roles: ADMIN , OPERATOR , FINANCE , CUSTOMER (with restrictions)

8.1 POST /api/bills/generate/{meterReadingId}

Description: Generate a bill from a meter reading. The system:

1. Validates the customer is **ACTIVE**
2. Finds the **active tariff** for the meter type
3. Calculates **consumption charge** (flat or tiered)
4. Adds the **active service charge**
5. Applies **tax** to (consumption + service charge)
6. Computes **total amount**
7. Sets status to **PENDING**
8. Sends an **email notification** to the customer
9. Inserts a **notification log** record

Required Role: ROLE_ADMIN or ROLE_OPERATOR

Path Parameters

Parameter	Description
meterReadingId	UUID of the captured meter reading

Success Response (200 OK)

```
{
  "id": "b8c9d0e1-f2a3-4567-bcde-890123456789",
  "billReference": "BILL-A3F7B2C1",
  "customerId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "customerName": "Marie Claire Mukamana",
  "meterId": "b2c3d4e5-f6a7-8901-bcde-f23456789012",
  "meterNumber": "WTR-2026-001",
  "meterReadingId": "c3d4e5f6-a7b8-9012-cdef-345678901234",
  "billingMonth": 5,
  "billingYear": 2026,
  "consumption": 130.25,
  "consumptionCharge": 39075.00,
  "serviceCharge": 500.00,
  "taxAmount": 7123.50,
  "penaltyAmount": 0.00,
  "totalAmount": 46698.50,
  "amountPaid": 0.00,
  "outstandingBalance": 46698.50,
  "status": "PENDING",
  "generatedAt": "2026-06-05T10:35:00",
  "approvedAt": null,
  "approvedById": null,
  "createdAt": "2026-06-05T10:35:00",
  "updatedAt": "2026-06-05T10:35:00"
}
```

Calculation Breakdown (Example)

Given:

- Consumption: **130.25 m³**
- Tariff: FLAT @ **300 FRW/unit**
- Service Charge: **500 FRW**
- Tax (VAT): **18%**

Consumption Charge	=	130.25 × 300	=	39,075.00	FRW
Service Charge	=	500.00			FRW
Taxable Base	=	39,075 + 500	=	39,575.00	FRW
Tax Amount	=	39,575 × 0.18	=	7,123.50	FRW
Total Amount	=	39,075 + 500 + 7,123.50	=	46,698.50	FRW

Error Response (400) - Inactive Customer

```
{
  "timestamp": "2026-06-05T10:35:00",
  "status": 400,
  "message": "Customer is inactive. Bills cannot be generated for inactive customers.",
  "path": "/api/bills/generate/c3d4e5f6-a7b8-9012-cdef-345678901234"
}
```

Error Response (400) - Missing Active Tariff

```
{
  "timestamp": "2026-06-05T10:35:00",
  "status": 400,
  "message": "No active tariff found for meter type: WATER",
  "path": "/api/bills/generate/c3d4e5f6-a7b8-9012-cdef-345678901234"
}
```

cURL Example

```
curl -X POST http://localhost:8080/api/bills/generate/c3d4e5f6-a7b8-9012-cdef-345678901234 \
-H "Authorization: Bearer <operator-token>"
```

8.2 GET /api/bills

Description: List all bills with pagination.

Required Role: `ROLE_ADMIN` or `ROLE_FINANCE`

8.3 GET /api/bills/{id}

Description: Get a single bill by ID.

Required Role: `ADMIN`, `FINANCE`, or `CUSTOMER` (own bills only)

CUSTOMER restriction: If the current user has `ROLE_CUSTOMER`, they can only view bills where `customerId` matches their linked customer profile. Attempting to view another customer's bill returns **403 Forbidden**.

8.4 PUT /api/bills/{id}/approve

Description: Approve a PENDING bill. Changes status to **APPROVED** .

Required Role: **ROLE_ADMIN** or **ROLE_FINANCE**

Success Response (200 OK)

Returns the bill with **status: "APPROVED"** and **approvedAt** timestamp.

Error Response (400) - Not Pending

```
{
  "timestamp": "2026-06-05T10:40:00",
  "status": 400,
  "message": "Only PENDING bills can be approved.",
  "path": "/api/bills/b8c9d0e1-f2a3-4567-bcde-890123456789/approve"
}
```

cURL Example

```
curl -X PUT http://localhost:8080/api/bills/b8c9d0e1-f2a3-4567-bcde-890123456789/approve \
-H "Authorization: Bearer <finance-token>"
```

8.5 GET /api/bills/customer/{customerId}

Description: List bills for a specific customer.

Required Role: **ADMIN** , **FINANCE** , or **CUSTOMER** (own customer ID only)

Query Parameters

Standard pagination: **page** , **size** , **sort** .

cURL Example

```
curl "http://localhost:8080/api/bills/customer/a1b2c3d4-e5f6-7890-abcd-ef1234567890?page=0&size=10"
-H "Authorization: Bearer <token>"
```

9. Payment Processing

Base URL: `/api/payments`
Required Role: `ROLE_FINANCE`

9.1 POST /api/payments

Description: Record a payment against a bill. Supports **partial payments**. When `outstandingBalance` reaches zero, the bill status becomes `PAID`, and a payment confirmation email is sent.

Payment Methods

- `CASH`
- `BANK_TRANSFER`
- `MOBILE_MONEY`

Request Body

```
{
  "billId": "b8c9d0e1-f2a3-4567-bcde-890123456789",
  "amountPaid": 20000.00,
  "paymentMethod": "MOBILE_MONEY",
  "paymentDate": "2026-06-10"
}
```

Validation Rules

Field	Rules
<code>billId</code>	Required, valid bill UUID
<code>amountPaid</code>	Required, > 0, must not exceed outstanding balance
<code>paymentMethod</code>	Required, <code>CASH</code> , <code>BANK_TRANSFER</code> , or <code>MOBILE_MONEY</code>
<code>paymentDate</code>	Required

Success Response (200 OK) - Partial Payment

```
{
  "id": "c9d0e1f2-a3b4-5678-cdef-901234567890",
  "billId": "b8c9d0e1-f2a3-4567-bcde-890123456789",
  "billReference": "BILL-A3F7B2C1",
  "amountPaid": 20000.00,
  "paymentMethod": "MOBILE_MONEY",
  "paymentDate": "2026-06-10",
  "reference": "PAY-8D3E2F1A",
  "recordedById": "d1e2f3a4-b5c6-7890-dabc-e12345678901",
  "createdAt": "2026-06-05T10:45:00",
  "updatedAt": "2026-06-05T10:45:00"
}
```

After this payment, the bill would show:

- **amountPaid** : 20,000.00
 - **outstandingBalance** : 26,698.50
 - **status** : **APPROVED** (not yet paid in full)
-

Full Payment Example (Bill becomes PAID)

If **amountPaid** equals the remaining **outstandingBalance** :

```
{
  "billId": "b8c9d0e1-f2a3-4567-bcde-890123456789",
  "amountPaid": 26698.50,
  "paymentMethod": "BANK_TRANSFER",
  "paymentDate": "2026-06-15"
}
```

Result:

- **status** becomes **PAID**
- **outstandingBalance** becomes **0.00**
- **Email sent** to customer: "Dear Marie, Your payment for bill BILL-A3F7B2C1 has been received. Outstanding balance: 0 FRW."
- **Notification log** inserted by DB trigger

Error Response (400) - Already Paid

```
{
  "timestamp": "2026-06-05T10:45:00",
  "status": 400,
  "message": "Bill is already fully paid.",
  "path": "/api/payments"
}
```

Error Response (400) - Overpayment

```
{
  "timestamp": "2026-06-05T10:45:00",
  "status": 400,
  "message": "Amount paid cannot exceed outstanding balance. Outstanding balance: 26698.50",
  "path": "/api/payments"
}
```

cURL Example

```
curl -X POST http://localhost:8080/api/payments \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <finance-token>" \
-d '{
  "billId": "b8c9d0e1-f2a3-4567-bcde-890123456789",
  "amountPaid": 20000.00,
  "paymentMethod": "MOBILE_MONEY",
  "paymentDate": "2026-06-10"
}'
```

9.2 GET /api/payments

Description: List all payments with pagination.

9.3 GET /api/payments/bill/{billId}

Description: Get all payments recorded for a specific bill.

cURL Example

```
curl http://localhost:8080/api/payments/bill/b8c9d0e1-f2a3-4567-bcde-890123456789 \
-H "Authorization: Bearer <finance-token>"
```

10. Notification Logs

Base URL: `/api/notifications`

Required Role: `ROLE_ADMIN` or `ROLE_FINANCE`

10.1 GET /api/notifications

Description: List all system-generated notification logs. Useful for verifying email delivery and DB trigger behavior.

Success Response (200 OK)

```
[
  {
    "id": "d0elf2a3-b4c5-6789-defa-012345678901",
    "customerId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "customerName": "Marie Claire Mukamana",
    "message": "Dear Marie Claire Mukamana, Your 5/2026 utility bill of 46698.50 FRW has been succe",
    "notificationType": "BILL_GENERATED",
    "createdAt": "2026-06-05T10:35:00",
    "sent": true
  },
  {
    "id": "elf2a3b4-c5d6-7890-efab-123456789012",
    "customerId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "customerName": "Marie Claire Mukamana",
    "message": "Dear Marie Claire Mukamana, Your payment for bill BILL-A3F7B2C1 has been received.",
    "notificationType": "PAYMENT_CONFIRMED",
    "createdAt": "2026-06-05T10:50:00",
    "sent": true
  }
]
```

10.2 GET /api/notifications/customer/{customerId}

Description: List notifications for a specific customer.

11. Complete Workflow Example

This step-by-step example demonstrates the full lifecycle from customer registration to payment confirmation.

Step 1: Login as Admin

```
curl -X POST http://localhost:8080/api/auth/login \
-H "Content-Type: application/json" \
-d '{
  "email": "admin@wasac.rw",
  "password": "Admin123"
}'
```

Save the token from the response for all subsequent calls.

Step 2: Create a Customer

```
curl -X POST http://localhost:8080/api/customers \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <admin-token>" \
-d '{
  "fullNames": "Jean Bosco Ndayisaba",
  "nationalId": "119907008765432",
  "email": "jean.ndayisaba@example.com",
  "phoneNumber": "+250788111222",
  "address": "KK 321 Ave, Kigali"
}'
```

Save the returned customer ID.

Step 3: Register a Meter

```
curl -X POST http://localhost:8080/api/meters \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <admin-token>" \
-d '{
  "meterNumber": "WTR-2026-002",
  "meterType": "WATER",
  "installationDate": "2026-01-20",
  "customerId": "<customer-id-from-step-2>"
}'
```

Save the returned meter ID.

Step 4: Create Tariff (FLAT)

```
curl -X POST http://localhost:8080/api/tariffs \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <admin-token>" \
-d '{
  "meterType": "WATER",
  "tariffType": "FLAT",
  "unitPrice": 350.00,
  "effectiveDate": "2026-01-01",
  "description": "Water flat rate 2026"
}'
```

Save the returned tariff ID.

Step 5: Activate the Tariff

```
curl -X PUT http://localhost:8080/api/tariffs/<tariff-id>/activate \
-H "Authorization: Bearer <admin-token>"
```

Step 6: Create Service Charge

```
curl -X POST http://localhost:8080/api/service-charges \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <admin-token>" \
-d '{
  "name": "Fixed Service Fee",
  "amount": 450.00,
  "meterType": "WATER",
  "effectiveDate": "2026-01-01"
}'
```

Step 7: Create Tax Config (VAT 18%)

```
curl -X POST http://localhost:8080/api/tax-configs \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <admin-token>" \
-d '{
  "taxName": "VAT",
  "rate": 0.18,
  "effectiveDate": "2026-01-01"
}'
```

Step 8: Capture Meter Reading (as Operator)

The admin token from Step 1 already includes `ROLE_OPERATOR`, so you can use it directly. In production, an operator would log in separately.

```
curl -X POST http://localhost:8080/api/meter-readings \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <admin-token>" \
-d '{
  "meterId": "<meter-id-from-step-3>",
  "previousReading": 500.00,
  "currentReading": 720.50,
  "readingDate": "2026-05-31",
  "readingMonth": 5,
  "readingYear": 2026
}'
```

Save the returned meter reading ID.

Step 9: Generate Bill

```
curl -X POST http://localhost:8080/api/bills/generate/<meter-reading-id> \
-H "Authorization: Bearer <admin-token>"
```

Expected Bill:

- Consumption: 220.50 m³
- Consumption Charge: $220.50 \times 350 = 77,175.00$ FRW
- Service Charge: **450.00 FRW**
- Taxable: $77,175 + 450 = 77,625.00$
- Tax (18%): **13,972.50 FRW**
- **Total: 91,597.50 FRW**

Side effects:

- Email sent to `jean.ndayisaba@example.com`
- Notification log inserted
- DB trigger `trg_after_bill_insert` fires

Step 10: Approve Bill (as Finance)

```
curl -X PUT http://localhost:8080/api/bills/<bill-id>/approve \
-H "Authorization: Bearer <admin-token>"
```

Step 11: Record Payment (as Finance)

```
curl -X POST http://localhost:8080/api/payments \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <admin-token>" \
-d '{
  "billId": "<bill-id-from-step-9>",
  "amountPaid": 91597.50,
  "paymentMethod": "MOBILE_MONEY",
  "paymentDate": "2026-06-05"
}'
```

Result:

- Bill status becomes **PAID**
- Outstanding balance: **0.00 FRW**
- Payment confirmation email sent

- DB trigger `trg_after_payment_update` fires
- Notification log inserted

Step 12: Verify Notifications

```
curl http://localhost:8080/api/notifications \
-H "Authorization: Bearer <admin-token>"
```

You should see two entries:

1. `BILL_GENERATED` — from the bill creation
2. `PAYMENT_CONFIRMED` — from the full payment (triggered by DB trigger)

12. Error Reference

HTTP Status Codes

Code	Meaning	Common Causes
200	OK	Successful GET, POST, PUT
204	No Content	Successful DELETE (soft-delete)
400	Bad Request	Validation errors, business rule violations, duplicate keys
401	Unauthorized	Missing or invalid JWT token
403	Forbidden	Valid token but insufficient role (e.g., customer accessing another's bill)
404	Not Found	Entity does not exist (wrong UUID)
500	Internal Server Error	Unexpected system error

Error Response Format

All errors return a consistent JSON structure:

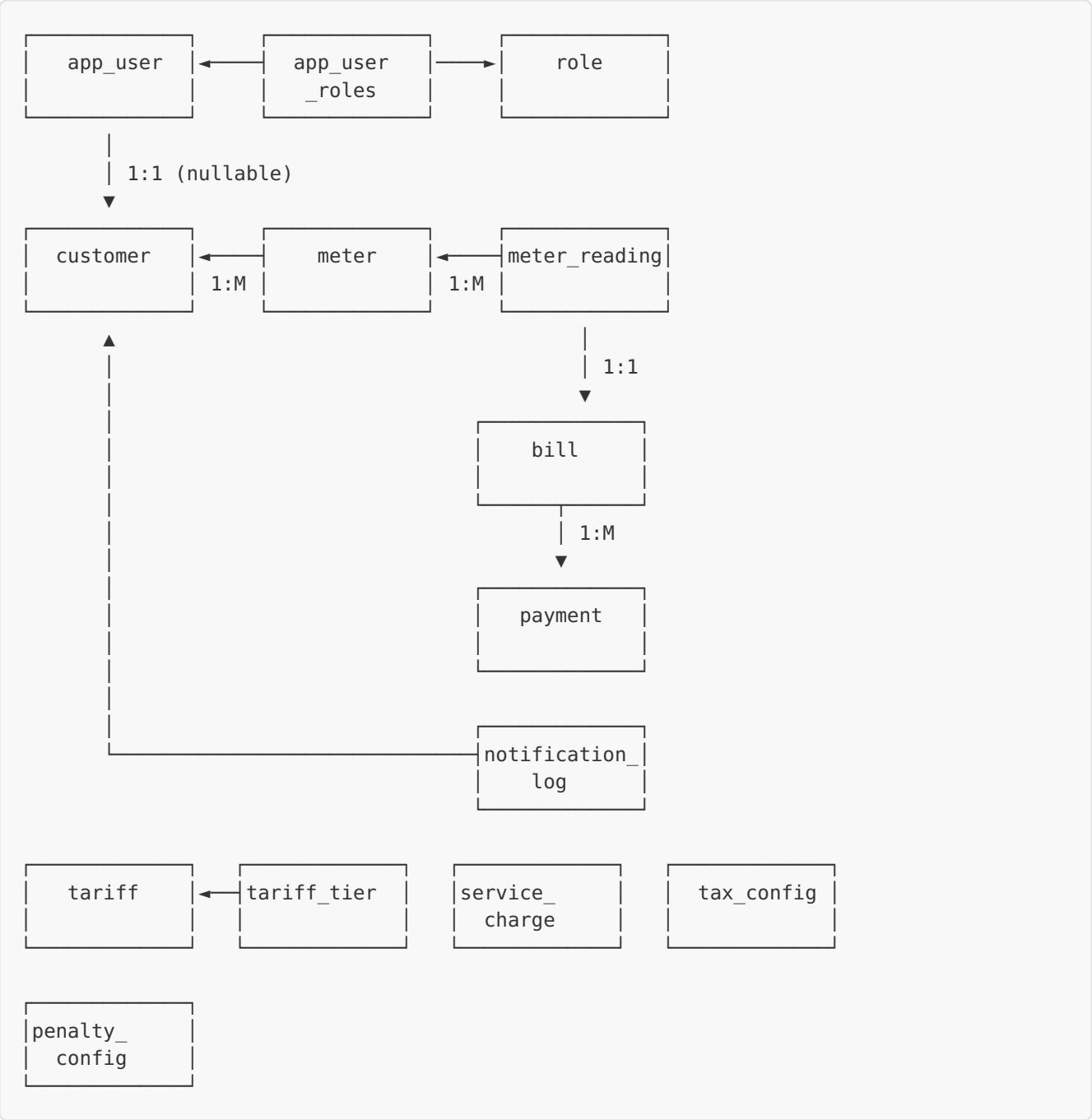
```
{
  "timestamp": "2026-06-05T10:00:00",
  "status": 400,
  "message": "Human-readable error description",
  "path": "/api/endpoint/that/failed"
}
```

Common Error Messages

Message	Endpoint	Fix
Email already in use	POST /api/auth/register	Use a different email
Customer with national ID already exists	POST /api/customers	Use a unique national ID
Meter number already exists	POST /api/meters	Use a unique meter number
Meter is not active	POST /api/meter-readings	Activate the meter first
Current reading must be strictly greater than previous reading	POST /api/meter-readings	Ensure current > previous
A reading for this meter already exists for the specified month and year	POST /api/meter-readings	Use a different month/year
No active tariff found for meter type	POST /api/bills/generate/{id}	Create and activate a tariff
Only PENDING bills can be approved	PUT /api/bills/{id}/approve	Bill is already approved or paid
Bill is already fully paid	POST /api/payments	Bill has no outstanding balance
Amount paid cannot exceed outstanding balance	POST /api/payments	Reduce amount to match balance
Customer is inactive	POST /api/bills/generate/{id}	Re-activate the customer first

13. Database Schema

Entity Relationship Overview



Key Constraints

Table	Constraint	Details
app_users	email	UNIQUE
customers	national_id	UNIQUE
customers	user_id	UNIQUE (nullable)
meters	meter_number	UNIQUE
meter_readings	(meter_id, month, year)	UNIQUE
bills	bill_reference	UNIQUE
bills	meter_reading_id	UNIQUE
payments	reference	UNIQUE

Database Triggers (PostgreSQL)

Trigger: `trg_after_bill_insert`

- **Fires:** After INSERT on `bills`
- **Action:** Calls `generate_bill_notification(NEW.id)`
- **Result:** Inserts `BILL_GENERATED` notification log

Trigger: `trg_after_payment_update`

- **Fires:** After UPDATE on `bills`
- **Condition:** `NEW.outstanding_balance = 0 AND OLD.outstanding_balance > 0`
- **Action:** Calls `on_full_payment(NEW.id)`
- **Result:** Updates status to `PAID`, inserts `PAYMENT_CONFIRMED` notification log

14. Testing with Swagger UI

14.1 Access Swagger UI

Open your browser and navigate to:

http://localhost:8080/swagger-ui.html

14.2 Authenticate in Swagger

1. **Expand** the **Authentication** section
2. Click **POST /api/auth/login → Try it out**
3. Enter the admin credentials:

```
json { "email": "admin@wasac.rw", "password": "Admin123" }
```
4. Click **Execute**
5. Copy the **token** value from the response
6. Click the **Authorize** button (green padlock icon) at the top of the page
7. Enter: **Bearer eyJhbGciOiJIUzI1NiIs...** (paste your full token)
8. Click **Authorize**, then **Close**

14.3 Test Endpoints

Now all protected endpoints are unlocked. You can:

- Click **Try it out** on any endpoint
- Fill in the request body (Swagger shows the schema)
- Click **Execute** to see the response

14.4 Swagger UI Endpoint Groups

Group	Description	Test Priority
Authentication	Login and register	Test first to get token
Customer Management	Create/view customers	Test second
Meter Management	Register meters	Test third
Tariff & Configuration	Pricing rules	Test fourth
Meter Reading Capture	Capture readings	Test fifth
Billing Engine	Generate and approve bills	Test sixth
Payment Processing	Record payments	Test seventh
Notification Logs	View audit trail	Test last

15. Environment Variables

Variable	Required	Default	Purpose
DB_USERNAME	No	postgres	PostgreSQL username
DB_PASSWORD	No	root	PostgreSQL password
JWT_SECRET	No	42371bdd...	JWT signing key (must be 256+ bits)
RESEND_API_KEY	No	re_QeFD8Cq2...	Resend email API key

Customizing for Production

Create a `application-prod.properties` or set environment variables:

```
export DB_USERNAME=utility_prod_user
export DB_PASSWORD=super_secure_password_123
export JWT_SECRET=your_256_bit_secret_key_here_minimum_32_chars
export RESEND_API_KEY=re_your_actual_production_key_here
```

Run with the production profile:

```
mvn spring-boot:run -Dspring-boot.run.profiles=prod
```

Appendix A: Tier Tariff Calculation Example

For a TIER tariff with these bands:

Tier	Min Units	Max Units	Price/Unit
1	0	50	200 FRW
2	51	100	250 FRW
3	101	null (unlimited)	300 FRW

For a consumption of **130 units**:

Tier 1: 50 units × 200 = 10,000 FRW
Tier 2: 50 units × 250 = 12,500 FRW (units 51-100)
Tier 3: 30 units × 300 = 9,000 FRW (units 101-130)

Total Consumption Charge: 31,500 FRW

Appendix B: Email Notifications

The system sends automated emails via **Resend**:

Bill Generation Email

```
<p>Dear Jean Bosco Ndayisaba,</p>
<p>Your 5/2026 utility bill of 91597.50 FRW has been successfully processed.</p>
```

Payment Confirmation Email

```
<p>Dear Jean Bosco Ndayisaba,</p>
<p>Your payment for bill BILL-A3F7B2C1 has been received. Outstanding balance: 0 FRW.</p>
```

Sender: Utility Billing System <notifications@updates.daisy.now>

Note: If email delivery fails, the system logs the error and continues without crashing.

Appendix C: Data Seeding

On application startup, the `DataInitializer` automatically creates:

Roles

- `ROLE_ADMIN`
- `ROLE_OPERATOR`
- `ROLE_FINANCE`
- `ROLE_CUSTOMER`

Default Admin User

Email: admin@wasac.rw
Password: Admin123
Roles: ADMIN, OPERATOR, FINANCE

No additional setup is required to begin testing.

End of Document

For support or questions, refer to the Swagger UI at <http://localhost:8080/swagger-ui.html> or examine the source code under `com.utilitybilling`.